Managed vs Native code, Exceptii, Testarea codului

Alexandru Olteanu

Universitatea Politehnica Bucuresti
Facultatea de Automatică si Calculatoare, Departamentul Calculatoare
alexandru.olteanu@cs.pub.ro

OOP, 2018

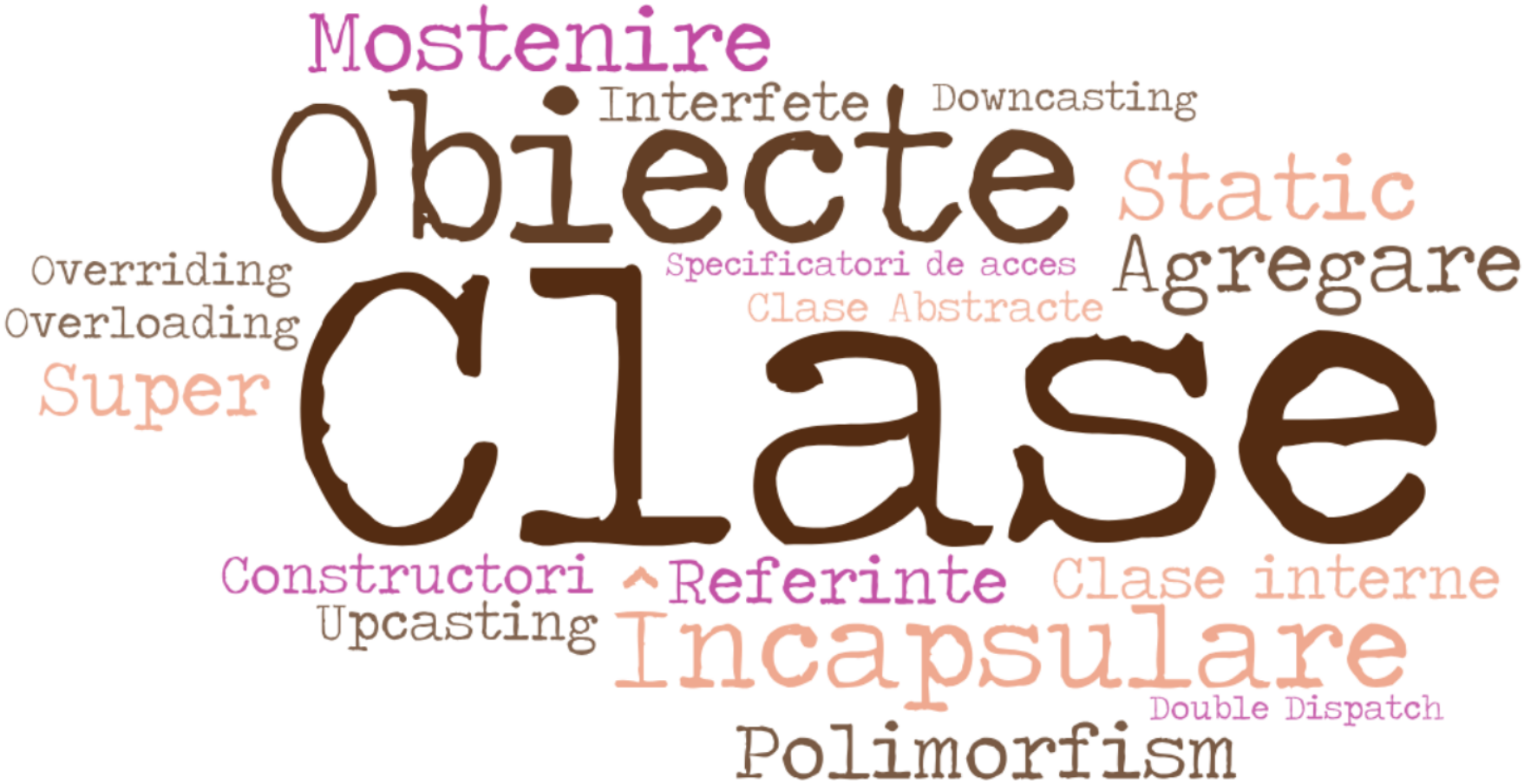






Universitatea
Politehnica
Bucureşti

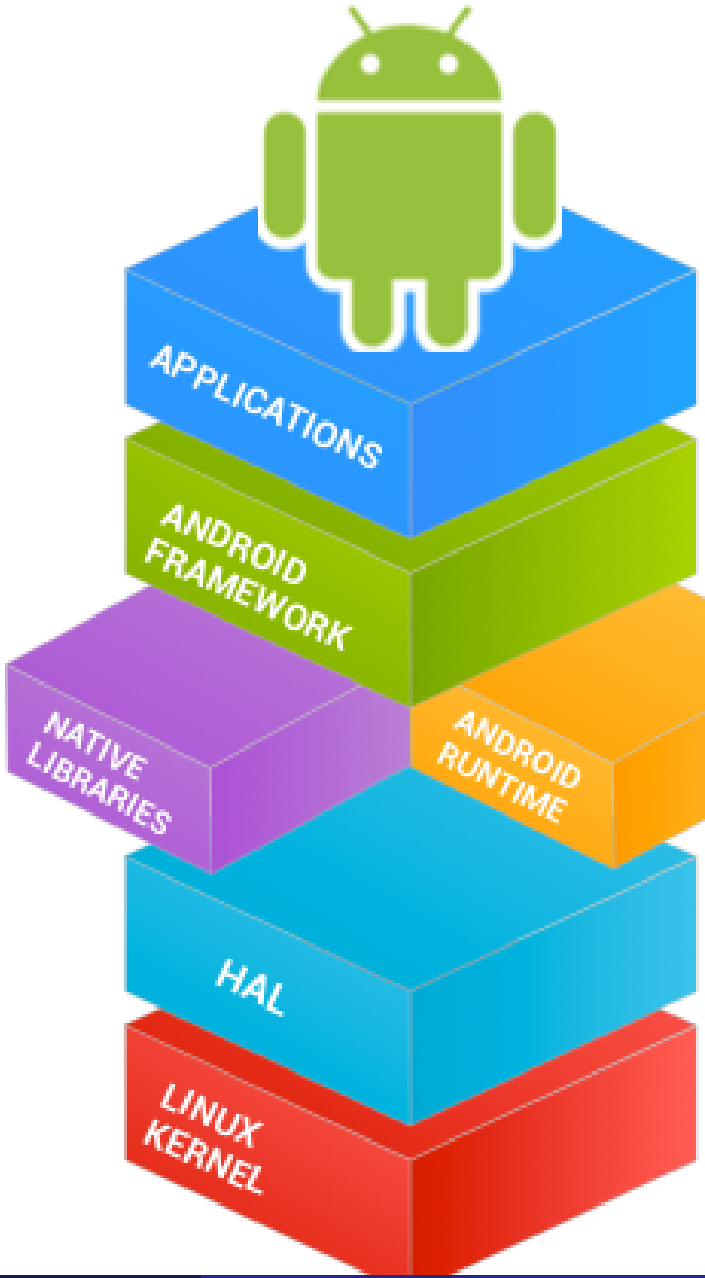
Partea 1: Bazele teoretice ale OOP



```
public interface Collection<E> extends Iterable<E> {
  boolean add(E e);
  boolean addAll(Collection<? extends E> c);
  <T> T[] toArray(T[] a);
}
```

Native code vs Managed code

Android Stack



ALARM • BROWSER • CALCULATOR • CALENDAR • CAMERA • CLOCK • CONTACTS • DIALER • EMAIL • HOME • IM • MEDIA PLAYER • PHOTO ALBUM • SMS/MMS • VOICE DIAL

CONTENT PROVIDERS • MANAGERS (ACTIVITY, LOCATION, PACKAGE, NOTIFICATION, RESOURCE, TELEPHONY, WINDOW) • VIEW SYSTEM

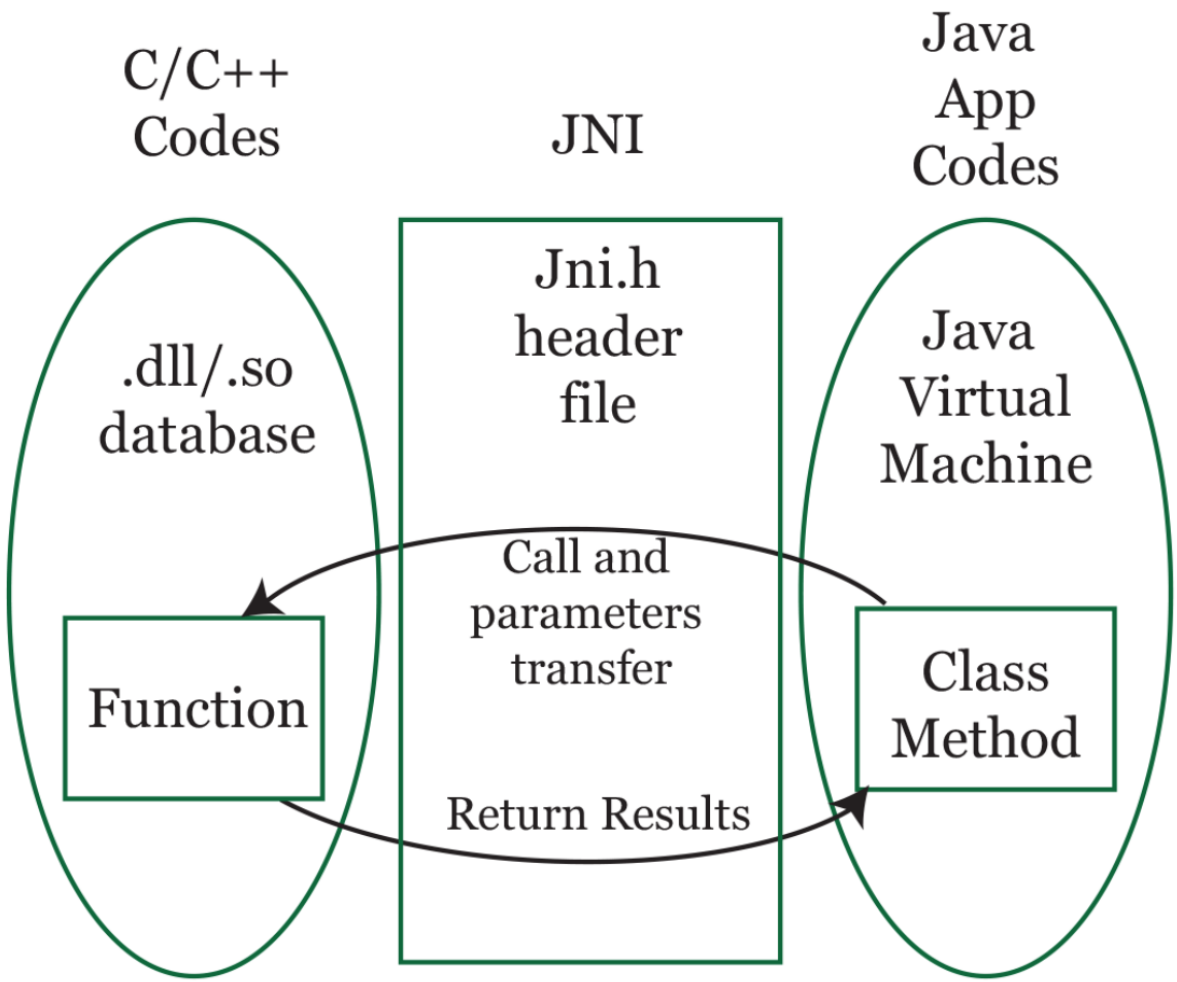
AUDIO MANAGER • FREETYPE • LIBC • MEDIA FRAMEWORK • OPENGL/ES • SQLITE • SSL • SURFACE MANAGER • WEBKIT

CORE LIBRARIES • ART • DALVIK VM

AUDIO • BLUETOOTH • CAMERA • DRM • EXTERNAL STORAGE • GRAPHICS • INPUT • MEDIA • SENSORS • TV

DRIVERS (AUDIO, BINDER (IPC), BLUETOOTH, CAMERA, DISPLAY, KEYPAD, SHARED MEMORY, USB, WIFI) • POWER MANAGEMENT

Java Native Interface



▸ Dr.Dobb's Android on x86: Java Native Interface and the Android Native Development Kit

Java Native Interface

Fisierul .h poate fi generat cu javah, e.g.:

```
cd src_path
javah -jni -classpath . com.edu.edumediaplayer.playback.
    MP3Reader
```

Exceptii

Conceptul de Exceptie

Programul nostru poate intampina situatii care nu corespund modului ideal de lucru. Spre exemplu:

- ar fi trebuit sa primesc un intreg, dar am primit o litera
- nu am gasit un fisier de intrare
- un alt program cu care comunic nu a mai raspuns la cereri
- am depasit dimensiunea structurii de date pe care am prevazut-o initial
- am incercat sa impart la zero etc.

Exceptie vs Eroare

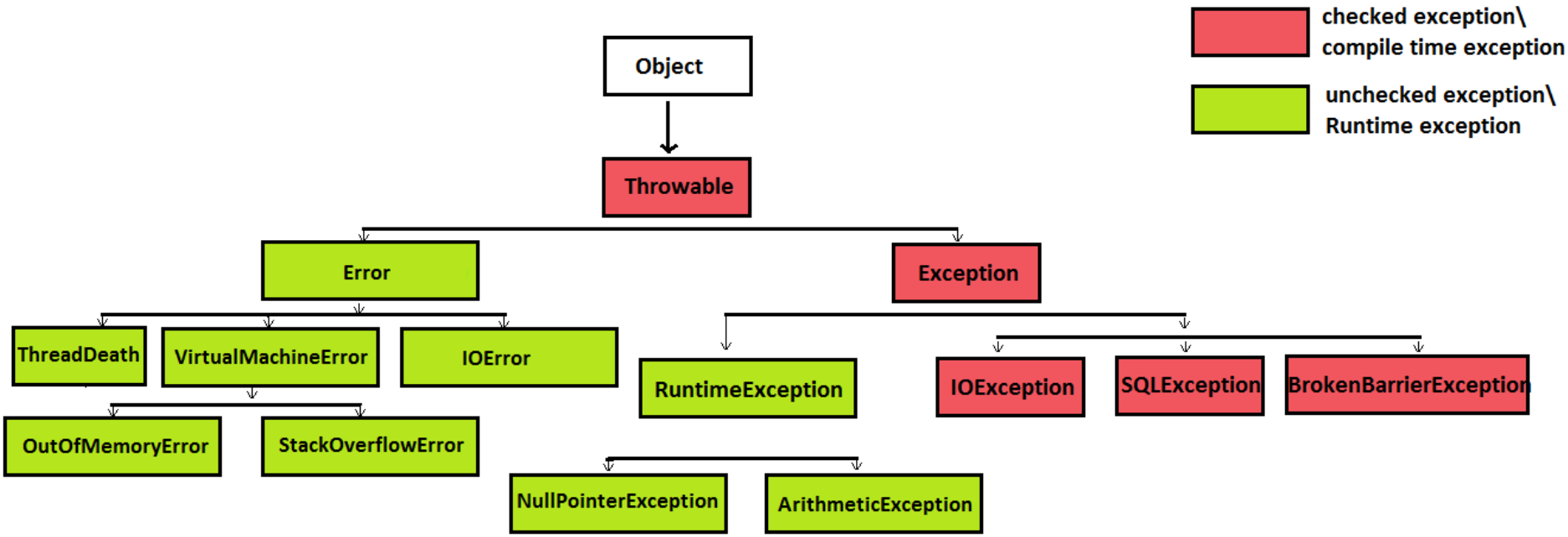
Pentru a trata in mod elegant si constructiv situatiile exceptionale, le vom modela prin intermediul unor clase de tip exceptie, care sa descrie situatia exceptionala.

Exceptie vs Eroare

Pentru a trata in mod elegant si constructiv situatiile exceptionale, le vom modela prin intermediul unor clase de tip exceptie, care sa descrie situatia exceptionala.

Erorile sunt situatii exceptionale atat de grave, incat nici nu ar trebui sa incercam sa le tratam.

Exceptii din Java



▸ Exceptions interview questions

Mecanismul de aruncare si prindere a exceptiilor



In stiva de apeluri:

- functia care detecteaza o exceptie si nu o poate trata, o arunca spre functia care a apelat-o
- sunt obligat sa tratez exceptiile checked (altfel eroare de compilare) dar daca nicio functie nu trateaza exceptiile unchecked programul se va intrerupe iremediabil.

Exceptii custom

```
public class ImproperSalaryException extends Exception {
    private int improperSalary;

    public ImproperSalaryException (String message, int
        salary) {
        super (message);
        improperSalary = salary;
    }

    public ImproperSalaryException (String message,
        Throwable cause) {
        super (message, cause);
    }
}
```

Aruncarea Exceptiei

```
public class Person {
    private int salary;
    public void setSalary(int salary) throws
        ImproperSalaryException {
        if (salary < 10000) {
            throw new ImproperSalaryException("Astia isi bat
                joc de mine", salary);
        }
    }
}
```

Prinderea Exceptiei

```
public class Person {
    private int salary;
    public void setSalary(String salary) throws
        ImproperSalaryException {
        try {
            int salary = Integer.parseInt(salary);
            setSalary(salary);
        } catch (NumberFormatException e) {
            throw new ImproperSalaryException("Au gresit in
                contract", e);
        }
    }
}
```

Prinderea Exceptiei

```
public class Manager {
    public static void main(String[] args) {
        Person Person = new Person();

        try {
            Person.setSalary(50000);
        } catch (ImproperSalaryException e) {
            if (e.getCause()==NULL) {
                Person.setSalary(10000);
            }
        }
    }
}
```

Tratarea Exceptiilor

Programul poate:

- trata local exceptia si continua executia
- comunica userului ca nu poate termina procesarea pe setul de date, dar poate trece la un alt set de date
- comunica userului ca nu poate continua executia, dar poate macar incerca sa salveze ce se poate salva

Tratarea Exceptiilor

Programul poate:

- trata local exceptia si continua executia
- comunica userului ca nu poate termina procesarea pe setul de date, dar poate trece la un alt set de date
- comunica userului ca nu poate continua executia, dar poate macar incerca sa salveze ce se poate salva

Atentie la exceptiile ce pot surveni in timpul tratarii exceptiilor: blocuri try-catch imbricate

Tratarea Exceptiilor

Programul poate:

- trata local exceptia si continua executia
- comunica userului ca nu poate termina procesarea pe setul de date, dar poate trece la un alt set de date
- comunica userului ca nu poate continua executia, dar poate macar incerca sa salveze ce se poate salva

Atentie la exceptiile ce pot surveni in timpul tratarii exceptiilor: blocuri try-catch imbricate
In catch putem decide ca nu este locul cel mai potrivit pentru tratarea exceptiei si o putem re-arunca mai sus in stiva de apeluri

Testarea codului

Unit Testing

Unit testing este o metoda de testare a software-ului prin care unitati individuale de cod (e.g. o clasa sau o metoda) impreuna cu datele si procedurile asociate sunt testate in vederea functionarii corecte.
Testele pot fi rulate in suita sau individual si pot fi rulate automat, spre exemplu de fiecare data cand programul este construit (build).

JUnit

JUnit este un cadru (framework) de testare, foarte popular in lumea dezvoltatorilor Java, bazat pe arhitectura xUnit.

▸ exemplu lab

Tutorial JUnit

▸ Un tutorial comprehensiv despre unit tests in Java

Highlights:

- teste cu JUnit
- Mocking and Stubbing
- Testing for Exceptions
- Integration Testing

Test doubles

Pentru a izola cat mai unitar partea programului care este testata:

- se poate analiza dependinta unitatilor de cod sub forma unei ierarhii, testand intai unitatile simple si apoi pe cele complexe care le folosesc
- se pot simula obiecte cu care unitatea testata interactioneaza, intr-un mod care sa garanteze ca erorile nu provin din aceste obiecte, folosind Mock

Test doubles

- Dummy objects are passed around but never actually used. Usually they are just used to fill parameter lists.
- Fake objects actually have working implementations, but usually take some shortcut which makes them not suitable for production (an in memory database is a good example).
- Stubs provide canned answers to calls made during the test, usually not responding at all to anything outside what's programmed in for the test. Stubs may also record information about calls, such as an email gateway stub that remembers the messages it 'sent', or maybe only how many messages it 'sent'.
- Mocks are what we are talking about here: objects pre-programmed with expectations which form a specification of the calls they are expected to receive.

Puteti citi mai multe pe ▸ Mocks aren't stubs

De citit

Este util sa cititi textul laboratorului inainte de laborator:

- Lab 5: Clase interne